

UNIVERSIDADE FEDERAL DE ALAGOAS - INSTITUTO DE COMPUTAÇÃO
PROJETO DE PROGRAMAÇÃO 2

RELATÓRIO DO MILESTONE 2

JACKUT - USER STORIES US5.1 A US9.2

Descrição arquitetural, refatoramentos e decisões de implementação para comunidades, mensagens, relacionamentos e remoção de usuários.

Documento técnico de implementação

Escopo: US5.1 - US9.2

SUMÁRIO

- | | |
|---|--|
| 1. Escopo do milestone | 6. US7.1 e US7.2 - Mensagens de comunidade |
| 2. Avaliação e critérios arquiteturais | 7. US8.1 e US8.2 - Relacionamentos |
| 3. Visão arquitetural da etapa | 8. US9.1 e US9.2 - Remoção de conta |
| 4. US5.1 e US5.2 - Comunidades | 9. Persistência e consistência |
| 5. US6.1 e US6.2 - Lista de comunidades | 10. Padrões e princípios aplicados |
| | 11. Conclusão |

1. ESCOPO DO MILESTONE

Este relatório registra exclusivamente o conjunto de alterações realizadas a partir das User Stories US5.1 até US9.2. Não é uma descrição integral do Jackut. Os componentes anteriores de cadastro, perfil, autenticação e recados privados são mencionados somente quando constituem dependências diretas das funcionalidades implementadas neste recorte.

A etapa introduziu quatro eixos de evolução: comunidades e suas consultas; lista de comunidades por usuário; mensagens de comunidade sobre uma infraestrutura de chat extensível; e um modelo genérico de relacionamentos que abrange amizade, fã, paquera e inimizade. Por fim, a remoção de conta passou a coordenar a eliminação dos dados associados ao usuário, incluindo seus vínculos, comunidades, chats, notificações e perfil.

User Story	Entrega principal	Impacto arquitetural
US5.1 - US5.2	Criação e consulta de comunidades.	Novo agregado de comunidade, repositório indexado por nome e controlador próprio.
US6.1 - US6.2	Vínculo e consulta rápida de comunidades por usuário.	Modelo <code>CommunityList</code> e índice por identificador de usuário.
US7.1 - US7.2	Mensagens destinadas a comunidades.	Generalização do chat para vários participantes, filas separadas e integrador de mensagens.
US8.1 - US8.2	Amizade, fã, paquera e inimizade.	Hierarquia polimórfica <code>Relationship</code> , enumeração de tipos e <code>State</code> para amizade.
US9.1 - US9.2	Remoção de usuário e persistência da remoção.	Controlador orquestrador e integradores responsáveis pela limpeza de cada domínio.

2. AVALIAÇÃO E CRITÉRIOS ARQUITETURAIS

A evolução foi conduzida com uma preocupação central: ampliar o domínio sem transformar a fachada em depósito de regras de negócio. A fachada pública deve expor a operação requerida pelos testes de aceitação e encaminhá-la a um controlador. Controladores coordenam casos de uso; serviços tratam regras próprias do domínio; repositórios preservam e recuperam entidades; integradores fazem a ponte entre domínios quando uma operação não pertence a um único agregado.

O principal risco técnico desta etapa era a duplicação de fluxos: chats privados e chats de comunidade partilham a entidade de chat, mas não possuem a mesma semântica de notificação e leitura. Outro risco era fazer com que as novas formas de relacionamento fossem implementadas por condicionais repetidas. As decisões adotadas privilegiam extensibilidade por composição e polimorfismo.

Critério de responsabilidade: uma classe que possui dados e operações necessárias para uma regra deve ser responsável por ela. Assim, `ChatUserState` controla suas filas de leitura, `Community` controla membros, `Relationship` controla estados e os repositórios mantêm índices consistentes com a persistência.

Limite deliberado: o mecanismo de remoção usa as filas de não lidas como fonte de verdade para mensagens ainda entregáveis. O modelo `Message` não armazena remetente, portanto a remoção específica de histórico já lido por autor não é inferível sem ampliar a entidade e o contrato de persistência.

3. VISÃO ARQUITETURAL DA ETAPA

O arranjo geral mantém uma arquitetura em camadas. `Facade` é o ponto de entrada externo. `CommunityController`, `UserController`, `ChatMessengerController` e `UserDeletionController` representam os casos de uso. Serviços implementam regras e repositórios persistem mapas de entidades em XML. Os integradores evitam que um controlador acesse repositórios de outro domínio diretamente.

Dois integradores são especialmente relevantes. `MessageBoxIntegrator` coordena caixas de mensagem, chat e mensagens: ele cria chats de comunidade, inclui participantes, envia notificações e remove dados de mensagem de um usuário. `InteractionIntegrator` centraliza a regra que bloqueia interação com quem marcou o remetente como inimigo, preservando a mensagem de erro baseada no nome exibido do perfil.

4. US5.1 E US5.2 - COMUNIDADES

4.1 Estrutura criada

A entidade `Community` representa uma comunidade por meio de identificador, nome único, descrição, dono, lista de membros e identificador do chat associado. O dono é incluído como primeiro membro no construtor. A presença simultânea de IDs e logins de membros permite persistência por identificador e construção do formato textual requerido pelos testes.

`CommunityRepository` estende `AbstractRepository<Community>`. Além do mapa principal por ID, mantém `communityByName`, um `HashMap<String, String>` que conecta nome ao ID. Esse índice evita percorrer todas as comunidades em consultas por nome, operação recorrente em descrição, dono, membros, entrada e envio de mensagens.

4.2 Fluxo de criação

1. `CommunityController.createCommunity` valida o usuário dono por `UserIntegrator`.
2. `CommunityService` verifica a disponibilidade do nome.
3. `MessageBoxIntegrator` constrói o chat inicial com o dono como participante.
4. `CommunityService` cria e persiste a entidade, recebendo o ID do chat.
5. `CommunityListService` vincula a comunidade à lista pessoal do dono.

O chat é criado antes da comunidade porque seu identificador é um dado obrigatório da entidade. Caso a validação de nome falhe, nenhum chat é criado. Essa ordenação reduz a possibilidade de chats órfãos.

4.3 Entrada de membros e consultas

Em `addCommunity`, o controlador resolve o login, pede ao serviço que inclua o membro na comunidade e usa o ID do chat retornado para adicionar o participante ao chat. A mesma operação inclui o nome da comunidade na `CommunityList` do usuário. Consultas de descrição, dono e membros permanecem em `CommunityService`, evitando que a fachada navegue pelo repositório.

4.4 Diagrama de classes da US5

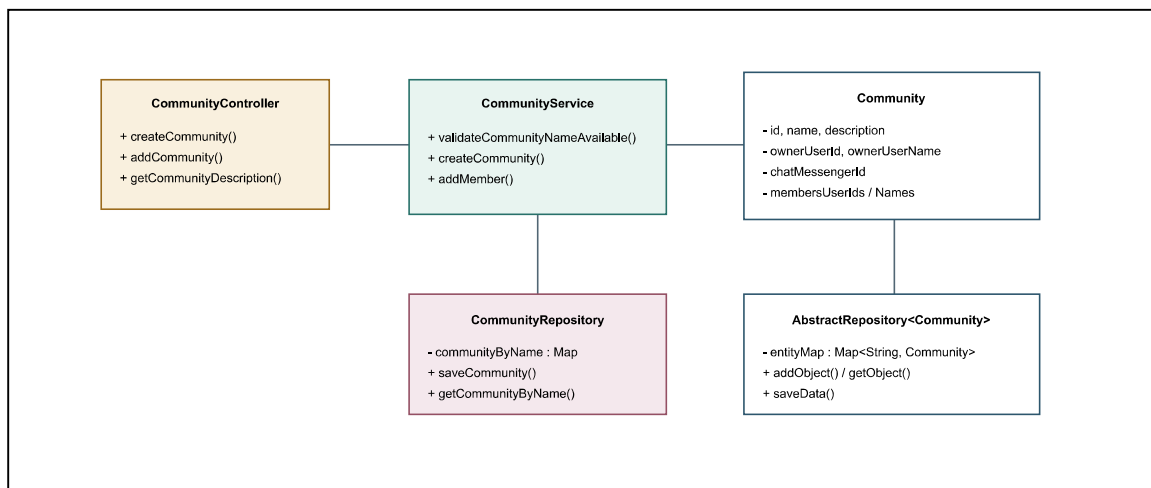


Figura 1 - Diagrama de classes da US5: criação, manutenção e consulta de comunidades.

Fonte: elaborado pelo autor.

5. US6.1 E US6.2 - LISTA DE COMUNIDADES POR USUÁRIO

A lista de comunidades foi modelada como entidade própria, e não como uma coleção colocada diretamente em `User`. A decisão separa as responsabilidades de autenticação e dados de conta das associações com comunidades, além de permitir persistência independente em `communityList.xml`.

`CommunityListRepository` mantém o mapa principal e o índice `communityListByUserId`. Com isso, `getCommunities` resolve a lista de um usuário em acesso direto. O método `buildCommunityList` concentra a formatação esperada pelo contrato de aceitação, retornando valores como `{UFEG, Computacao}`.

Componente	Responsabilidade	Razão da separação
<code>CommunityList</code>	Guardar e formatar nomes de comunidades do usuário.	Entidade coesa para uma associação de muitos valores.
<code>CommunityListService</code>	Criar lista, adicionar comunidade e consultar nomes.	Centraliza validação contra duplicação de vínculo.
<code>CommunityListRepository</code>	Persistir lista e consultar por usuário.	Índice dedicado elimina buscas lineares.

5.1 Diagrama de classes da US6

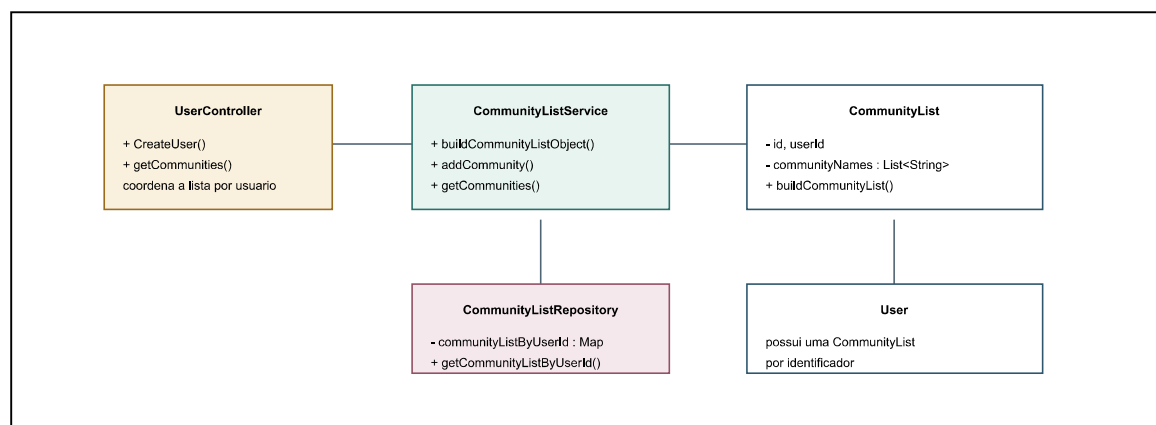


Figura 2 - Diagrama de classes da US6: associação e consulta de comunidades por usuário. Fonte: elaborado pelo autor.

6. US7.1 E US7.2 - MENSAGENS DE COMUNIDADE

6.1 Chat preparado para múltiplos participantes

Embora o chat privado original trabalhasse com dois participantes, `ChatMessenger` foi refatorado para conter `Map<String, ChatUserState> messengerStates`. Cada chave corresponde a um participante e cada valor controla as mensagens lidas e não lidas daquele usuário. O método `addParticipant` inclui incrementalmente um membro e cria seu estado de leitura sem alterar os estados existentes.

`sendMessage` entrega a mensagem a todos exceto ao remetente, comportamento de recado privado. Já `sendMessageToAll` registra a mensagem para todos os participantes, incluindo o autor, comportamento adotado para o feed de comunidade. Essa distinção foi mantida na entidade de chat, evitando duplicação de estruturas de conversa.

6.2 Filas de notificação separadas

`MessengerBox` passou a manter duas filas: `messengerNotifications` para recados privados e `communityMessageNotifications` para mensagens de comunidades. Assim, `lerRecado` e `lerMensagem` preservam contratos diferentes, embora ambos usem o ID da mensagem como notificação.

No envio de uma mensagem de comunidade, `MessageBoxIntegrator.sendMessage` cria `Message`, solicita que `ChatMessengerService` a distribua aos participantes e alimenta a fila de comunidade de cada caixa. Na leitura, a notificação aponta para a mensagem, a mensagem aponta para o chat e o `ChatUserState`

Escalabilidade conceitual: a comunidade usa o mesmo mecanismo preparado para múltiplos usuários. Adicionar novos membros não requer criar outro chat, nem replicar mensagens por par de usuários; basta inserir outro `ChatUserState` no mapa do chat.

6.3 Diagrama de classes da US7

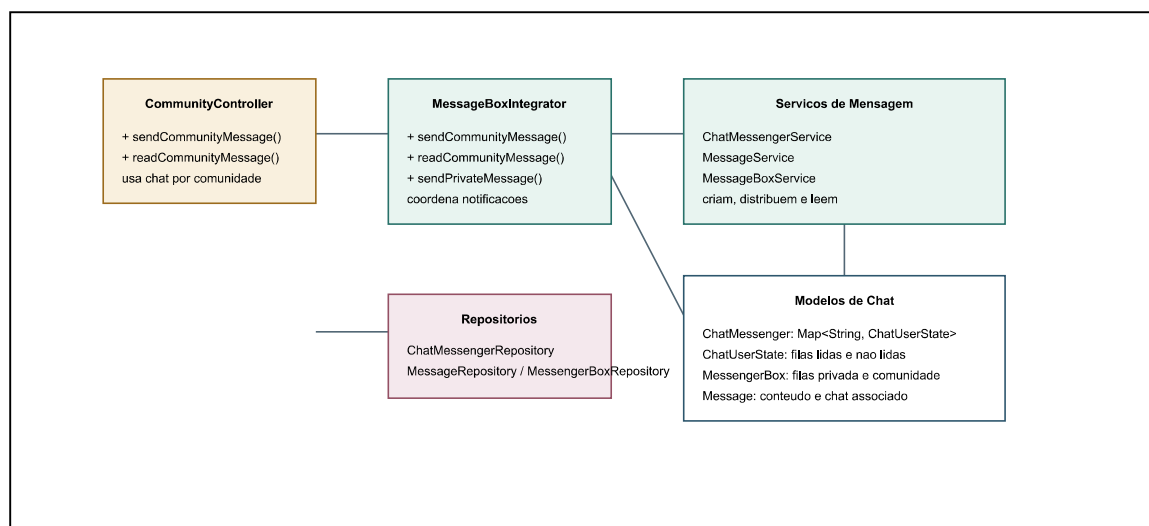


Figura 3 - Diagrama de classes da US7: distribuição, notificação e leitura de mensagens em comunidades. Fonte: elaborado pelo autor.

7. US8.1 E US8.2 - RELACIONAMENTOS

7.1 Generalização por polimorfismo

O modelo anterior de amizade foi generalizado em `Relationship`, uma superclasse abstrata. As classes `FriendshipRelationship`, `FanRelationship`, `CrushRelationship` e `EnemyRelationship` especializam apenas o tipo sem replicar armazenamento e manipulação de estados. A enumeração `RelationshipType` contém `FRIENDSHIP`, `FAN`, `CRUSH` e `ENEMY`, além de termos usados na composição de mensagens de exceção.

O repositório armazena todas as instâncias pelo tipo abstrato `Relationship`. Com isso, não há dependência de `instanceof` para executar o fluxo principal. O tipo é um dado explícito e permite que serviços e repositórios localizem relações por `userId` e `RelationshipType`.

7.2 State para amizade

Amizade possui um ciclo que não existe nos outros tipos: ausência de relação, solicitação enviada, solicitação recebida e relação atual. Para não concentrar todas as transições em condicionais no serviço, foram criados `RelationshipStateHandle` e os manipuladores `NoRelationshipState`, `SentRequestRelationshipState`, `ReceivedRequestRelationshipState` e `CurrentRelationshipState`.

`RelationshipService` seleciona o manipulador pela situação do usuário e delega a transição. Em uma amizade comum, a primeira chamada cria solicitação; a chamada recíproca confirma a amizade; chamadas incompatíveis produzem as exceções específicas do domínio. Fã, paquera e inimigo usam o estado `CURRENT` sem ciclo de confirmação.

7.3 Paquera recíproca e bloqueio

A fachada não contém a regra de paquera recíproca. `UserController.addCrush` cria a relação, consulta a relação inversa e, se ambas existirem, delega os recados automáticos a `MessageBoxIntegrator`. A regra de bloqueio por inimizado foi extraída para `InteractionIntegrator`. Esse integrador consulta relacionamento, usuário e perfil para produzir `FuncaoInvalida` com o nome exibido correto. Ele é reutilizado por amizade, fã, paquera e envio de recado.

7.4 Diagrama de classes da US8

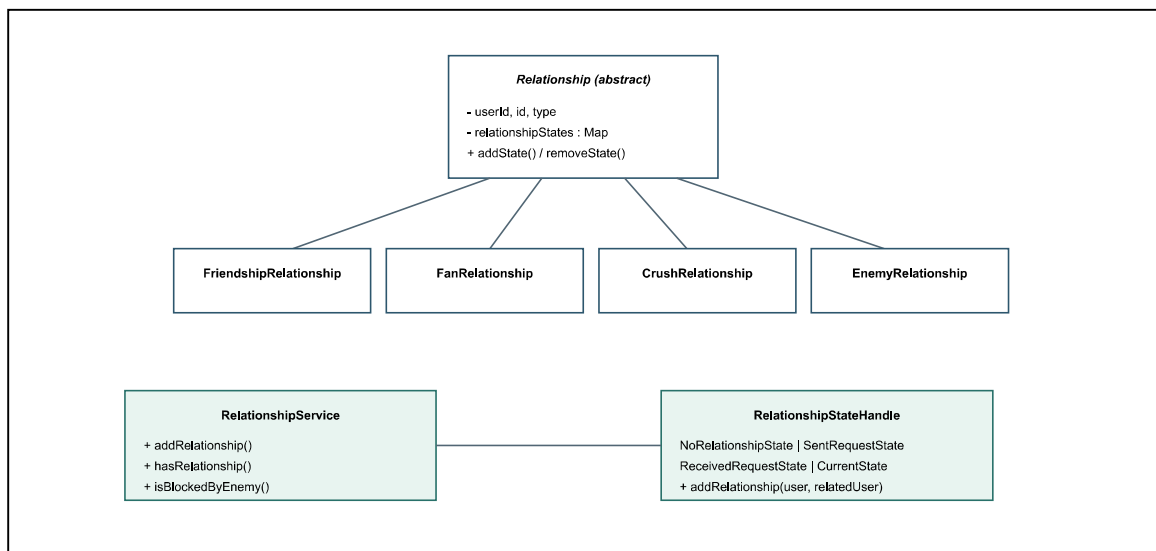


Figura 4 - Diagrama de classes da US8: tipos polimórficos de relacionamento e ciclo de estados da amizade. Fonte: elaborado pelo autor.

8. US9.1 E US9.2 - REMOÇÃO DE CONTA

8.1 Orquestração sem acoplamento indevido

`UserDeletionController` concentra a ordem do caso de uso `removerUsuario`, mas não conhece detalhes internos de persistência. Ele valida a existência do usuário e aciona integradores responsáveis por seus próprios conjuntos de dados: `RelationshipIntegrator`, `CommunityIntegrator`, `MessageBoxIntegrator`, `ProfileIntegrator` e `UserIntegrator`.

A ordem importa. Relações, comunidades e mensagens são limpas enquanto o usuário ainda pode ser resolvido pelos índices. Perfil e cadastro principal são removidos por último. Isso mantém as referências disponíveis durante a limpeza e faz com que uma segunda remoção falhe corretamente com `UsuarioNaoCadastrado`.

8.2 Comunidades e listas

`CommunityIntegrator.deleteUserCommunities` percorre comunidades por uma cópia de sua coleção. Quando o usuário é dono, a comunidade é removida do repositório e de todas as `CommunityList`. Quando é apenas membro, seu ID e login são removidos da comunidade e o chat é marcado para preservação, com remoção apenas do participante. A lista de comunidades do usuário removido é então excluída.

8.3 Chats, estados e mensagens não lidas

Para chats que deixam de existir, `ChatMessengerRepository.deleteChatsAndCollectUnreadMessages` coleta os IDs presentes nas filas `UnreadMessengers` de todos os `ChatUserState`. Esses IDs representam mensagens que ainda poderiam ser entregues. `MessageService` remove essas mensagens e `MessageBoxRepository` remove as notificações correspondentes de todas as caixas. Por fim, a caixa do usuário removido é excluída.

Em chats de comunidades preservadas, o usuário é removido do conjunto de participantes e de seu respectivo `ChatUserState`; os demais participantes continuam com seus estados. Isso garante que a saída de um membro não destrua a conversa da comunidade.

8.4 Diagrama de classes da US9

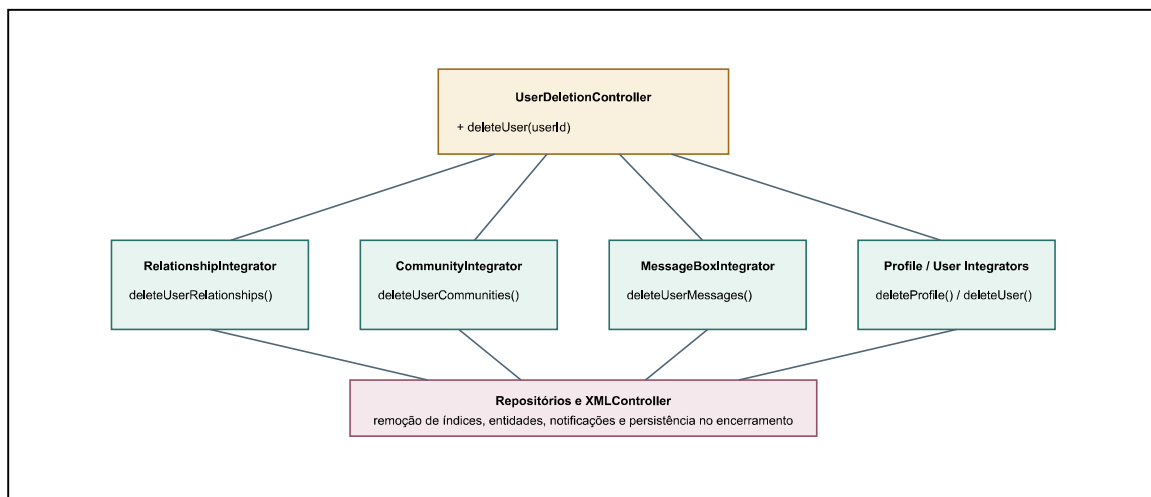


Figura 5 - Diagrama de classes da US9: orquestração da remoção de conta e limpeza de dependências. Fonte: elaborado pelo autor.

9. PERSISTÊNCIA E CONSISTÊNCIA

As novas entidades seguem o mecanismo de repositórios baseado em `AbstractRepository` e `XMLController`. Cada repositório mantém seu mapa principal em memória e o persiste no arquivo XML correspondente. O método `encerrarSistema` delega a persistência aos controladores existentes: usuário persiste usuários, perfis, relacionamentos, caixas e listas; chat persiste chats e mensagens; comunidade persiste comunidades.

Dados	Repositório	Índice auxiliar	Finalidade
Comunidades	<code>CommunityRepository</code>	<code>communityByName</code>	Consulta direta por nome.
Listas de comunidade	<code>CommunityListRepository</code>	<code>communityListByUserId</code>	Consulta direta por usuário.
Caixas de mensagem	<code>MessengerBoxRepository</code>	<code>messengerBoxByUserId</code>	Notificação e leitura por usuário.
Chats	<code>ChatMessengerRepository</code>	<code>chatMessengerList</code>	Reuso de chat por participantes.
Relacionamentos	<code>RelationshipRepository</code>	Por usuário e por tipo	Busca polimórfica e remoção consistente.

10. PADRÕES E PRINCÍPIOS APLICADOS

Padrão ou princípio	Aplicação no milestone	Justificativa
Facade	Facade expõe a API usada pelos testes.	Oferece uma entrada estável e evita que testes dependam da organização interna.
Repository	Repositórios de comunidade, lista, chat, mensagem e relacionamento.	Separa persistência e índices das regras de negócio.
Service Layer	CommunityService, RelationshipService, ChatMessengerService.	Concentra regras de domínio e reduz lógica nos controladores.
Integrator	Mensagens, interação, perfil, relacionamento e comunidade.	Coordena domínios sem expor repositórios entre controladores.
State	Manipuladores de estado da amizade.	Modela transições de solicitação e confirmação sem encadear condicionais no serviço.
Polimorfismo	Hierarquia Relationship.	Amplia tipos de relacionamento sem duplicar estado e sem instanceof.
Singleton	Instâncias compartilhadas de repositórios e alguns integradores.	Garante um ponto de acesso aos mapas em memória e aos índices persistidos.
Information Expert	ChatUserState, Community, Relationship.	Atribui comportamento a quem possui os dados necessários para executá-lo.

11. CONCLUSÃO

O milestone amplia o Jackut sem alterar o papel da fachada: novas operações continuam expostas por uma API simples, enquanto as regras permanecem distribuídas entre controladores, serviços, modelos, repositórios e integradores. Comunidades foram implementadas como entidades persistentes e preparadas para um chat com vários participantes. Relacionamentos passaram de um caso específico de amizade para uma família polimórfica de tipos, com State reservado ao ciclo que realmente exige transições.

A remoção de usuário consolidou a preocupação com consistência entre agregados. A operação não se limita a excluir o cadastro principal: limpa relações, listas, comunidades, participantes de chat, notificações pendentes, perfil e índices auxiliares. O resultado é uma base mais modular para futuras User Stories, especialmente as que expandirem mensagens, comunidade e histórico de conversa.

Relatório do Milestone 2 - Jackut. Escopo documentado: US5.1 a US9.2.